Chapter 5

The Relational Data Model and Relational Database Constraints

Chapter Outline

- Relational Model Concepts
- Relational Model Constraints and Relational Database Schemas
- Update Operations and Dealing with Constraint Violations

Relational Model Concepts

- The relational Model of Data is based on the concept of a ***Relation***.

- A Relation is a mathematical concept based on the ***ideas of sets***.

- The strength of the relational approach to data management comes from the formal foundation provided by the theory of relations.

E, STATUS, CITY)
0,KANDY>

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

Relational Model Concepts

- The model was first proposed by Dr. E.F. Codd of IBM in 1970 in the following paper:
  "A Relational Model for Large Shared Data Banks," Communications of the ACM, June 1970.

  The above paper caused a major revolution in the field of Database management and earned Ted Codd the coveted ACM Turing Award.

<u>INFORMAL</u> DEFINITIONS

- RELATION: A table of values
  - A relation may be thought of as a **set of rows**.
  - A relation may alternately be though of as a **set of columns**.
  - Each row represents a fact that corresponds to a real-world **entity** or **relationship**.
  - Each row has a value of an item or set of items that uniquely identifies that row in the table.
    - Sometimes row-ids or sequential numbers are assigned to identify the rows in the table.
  - Each column typically is called by its column name or column header or attribute name.

FORMAL DEFINITIONS

- A **Relation** may be defined in multiple ways.
- The **Schema** of a Relation: *R* (A1, A2, ...An)

  Relation schema *R* is defined over **attributes** A_1, A_2, ...A_n

  For Example -

  CUSTOMER (Cust-id, Cust-name, Address, Phone#)

  Here, CUSTOMER is a relation **defined over** the four attributes Cust-id, Cust-name, Address, Phone#, each of which has a **domain** or a set of valid values. For example, the domain of Cust-id is 6 digit numbers.

FORMAL DEFINITIONS

- A **tuple** is an ordered set of values
- Each value is derived from an appropriate **domain**.
- Each row in the CUSTOMER table may be referred to as a tuple in the table and would consist of four values.
- **<632895, "John Smith", "101 Main St. Atlanta, GA 30332", "(404) 894-2000">** is a tuple belonging to the CUSTOMER relation.
- A relation may be regarded as a ***set of tuples*** (rows).
- Columns in a table are also called **attributes** of the relation.
- The **degree** of a relation is the number of attributes **n** of its relation schema.

FORMAL DEFINITIONS

- A **domain** has a logical definition: e.g., "USA_phone_numbers" are the set of 10 digit phone numbers valid in the U.S.
- A domain may have a data-type or a format defined for it. The USA_phone_numbers may have a format: (ddd)-ddd-dddd where each d is a decimal digit. E.g., Dates have various formats such as monthname, date, year or yyyy-mm-dd, or dd mm,yyyy etc.
- An attribute designates the **role** played by the domain. E.g., the domain Date may be used to define attributes "Invoice-date" and "Payment-date".

FORMAL DEFINITIONS

- The relation is formed over the Cartesian product of the sets; each set has values from a domain; that domain is used in a specific role which is conveyed by the attribute name.
- For example, attribute Cust-name is defined over the domain of strings of 25 characters. The role these strings play in the CUSTOMER relation is that of the name of customers.
- Formally,

  Given $R(A_1, A_2, ..., A_n)$

  $r(R) \subset dom(A_1) \times dom(A_2) \times ....\times dom(A_n)$
- R: schema of the relation
- r of R: a specific "value" or population of R.
- R is also called the **intension** of a relation
- r is also called the **extension** of a relation
- PS/2018/234

FORMAL DEFINITIONS

- Let S1 = {0,1}
- Let S2 = {a,b,c}
- S1 X S2 = {<0,a>, <0,b>,<0,c>, <1,a>,<1,b>,<1,c>} ((2*3 =6)
- Let R $\subset$ S1 X S2
- Then for example: r(R) ={<0,a> , <0,b> , <1,c> } is one possible "state" or "population" or "extension" r of the relation R, defined over domains S1 and S2. It has three tuples.

R	
S1	S2
0	a
0	b
1	c

FORMAL DEFINITIONS

- SUPPLIER (SNO,SNAME,STATUS,CITY)
- SNO = {S1,S2,S3,S4,S5} (5)
- SNAME = {SAMAN,JAGATH,ANIL,GAMINI,BALA} (5)
- STATUS={20,10,30} (3)
- CITY={colombo,kandy,jaffna} (3)
- SUPPLIER $\subset$ SNO X SNAME X STATUS X CITY (5*5*3*3=225)

- Then for example: r(SUPPLIER)={<S1,SAMAN,20,COLOMBO>, <S2,JAGATH,10,KANDY>, <S3,ANIL,30,KANDY>, <S4,GAMINI,20,COLOMBO>, <S5,BALA,30,JAFFNA>} is one possible "state" or "population" or "extension" r of the relation SUPPLIER, defined over domains SNO, SNAME, STATUS and CITY. It has 5 tuples.

DEFINITION SUMMARY

Informal Terms		**Formal Terms**
Table		Relation
Column		Attribute/Domain
Row		Tuple
Values in a column		Domain
Table Definition		Schema of a Relation
Populated Table		Extension

Example - **Figure 5.1**

Relation name

Attributes

Tuples

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

CHARACTERISTICS OF RELATIONS

- **Ordering of tuples in a relation r(R)**: The tuples are *not* considered to be ordered, even though they appear to be in the tabular form.
- **Ordering of attributes in a relation schema R** (and of values within each tuple): We will consider the attributes in $R(A_1, A_2, ..., A_n)$ and the values in $t=<v_1, v_2, ..., v_n>$ to be *ordered* .

  (However, a more general *alternative definition* of relation does not require this ordering).
- **Values in a tuple**: All values are considered *atomic* (indivisible). A special **null** value is used to represent values that are unknown or inapplicable to certain tuples.

CHARACTERISTICS OF RELATIONS

- <u>Notation:</u>

- We refer to **component values** of a tuple t by $t[A_i] = v_i$ (the value of attribute A_i for tuple t).

  Similarly, $t[A_u, A_v, ..., A_w]$ refers to the subtuple of t containing the values of attributes $A_u, A_v, ..., A_w$, respectively.

CHARACTERISTICS OF RELATIONS- Figure 5.2

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21

Relational Database Schemas

- **Relational Database Schema**: A set S of relation schemas that belong to the same database. i.e. S = $\{R_1, R_2, ..., R_n\}$ and a set of **integrity constraints** IC
- {P,S,J,SPJ}
- A **relational database state** DB of S is a set of relation states, DB = $\{r_1, r_2,\ldots,r_n\}$ such that r_i is a state of R_i and r_i relation state satisfy the integrity constraints specified in IC.

- **Invalid state** – a database state does not obey all the integrity constraint.
- **Valid state** – a database state satisfies all the constraints in IC
- Integrity constraints are specified on a database schema and are expected to hold on every valid database state of that schema.

Relational Model Constraints

- Constraints on databases can generally be divided into three main categories:
  1. Constraints that are inherent in the data model. – **inherent model-based constraint.** eg. A relation cannot have duplicate tuples.
  2. Constraints that can be directly expressed in the schemas of the data model, by specifying them in the DDL. – **schema-based constraints.**
  3. Constraints that cannot be directly expressed in the schemas of the data model, and be expressed and by the application program. **– application-based constraints.**

- Constraints are *conditions* that must hold on *all* valid relation instances. These include:
  1. **Domain** constraints
  2. **Key** constraints
  3. **Entity integrity** constraints
  4. **Referential integrity** constraints

Domain Constraint

- Within each tuple, the value of each attribute A must be an atomic value from the domain dom(A).

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

Key Constraints

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

{ssn, name, age} – Super Key
{ssn} - Key
{sno}- Key

{s1,saman,colombo} <>
{s6, saman, colombo}

S			
SNO	SNAME	STATUS	CITY
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA
S6	SAMAN	10	Colombo

Key Constraints

- **<u>Superkey</u>** <u>of R:</u> A set of attributes SK of R such that no two tuples *in any valid relation instance r(R)* will have the same value for SK. That is, for any distinct tuples t1 and t2 in r(R), t1[SK] ≠ t2[SK].
- **<u>Key</u>** <u>of R:</u> A "minimal" superkey; that is, a superkey K such that removal of any attribute from K results in a set of attributes that is not a superkey.

<u>Example</u>: The CAR relation schema:

CAR(<u>State</u>, <u>Reg#</u>, SerialNo, Make, Model, Year)

has two keys Key1 = {State, Reg#}, Key2 = {SerialNo}, which are also superkeys. {SerialNo, Make} is a superkey but *not* a key.

Key Constraints

- In general a relation schema may have more than one key. In this case each of the keys is called **candidate keys**.
- If a relation has *several* **candidate keys**, one is chosen arbitrarily to be the **primary key**. The primary key attributes are *underlined*.

Key Constraints

Figure: The CAR relation with two candidate keys:

LicenseNumber and EngineSerialNumber

CAR	LicenseNumber	EngineSerialNumber	Make	Model	Year
	Texas ABC-739	A69352	Ford	Mustang	96
	Florida TVP-347	B43696	Oldsmobile	Cutlass	99
	New York MPO-22	X83554	Oldsmobile	Delta	95
	California 432-TFY	C43742	Mercedes	190-D	93
	California RSK-629	Y82935	Toyota	Camry	98
	Texas RSK-629	U028365	Jaguar	XJS	98

Entity Integrity

- **Entity Integrity**: The *primary key attributes* PK of each relation schema R in S cannot have null values in any tuple of r(R). This is because primary key values are used to *identify* the individual tuples.

$$t[PK] \neq \text{null for any tuple } t \text{ in } r(R)$$

- <u>Note:</u> Other attributes of R may be similarly constrained to disallow null values, even though they are not members of the primary key.

Entity Integrity

CAR	LicenseNumber	EngineSerialNumber	Make	Model	Year
	Texas ABC-739	A69352	Ford	Mustang	96
	Florida TVP-347	B43696	Oldsmobile	Cutlass	99
	New York MPO-22	X83554	Oldsmobile	Delta	95
	California 432-TFY	C43742	Mercedes	190-D	93
	California RSK-629	Y82935	Toyota	Camry	98
	Texas RSK-629	U028365	Jaguar	XJS	98

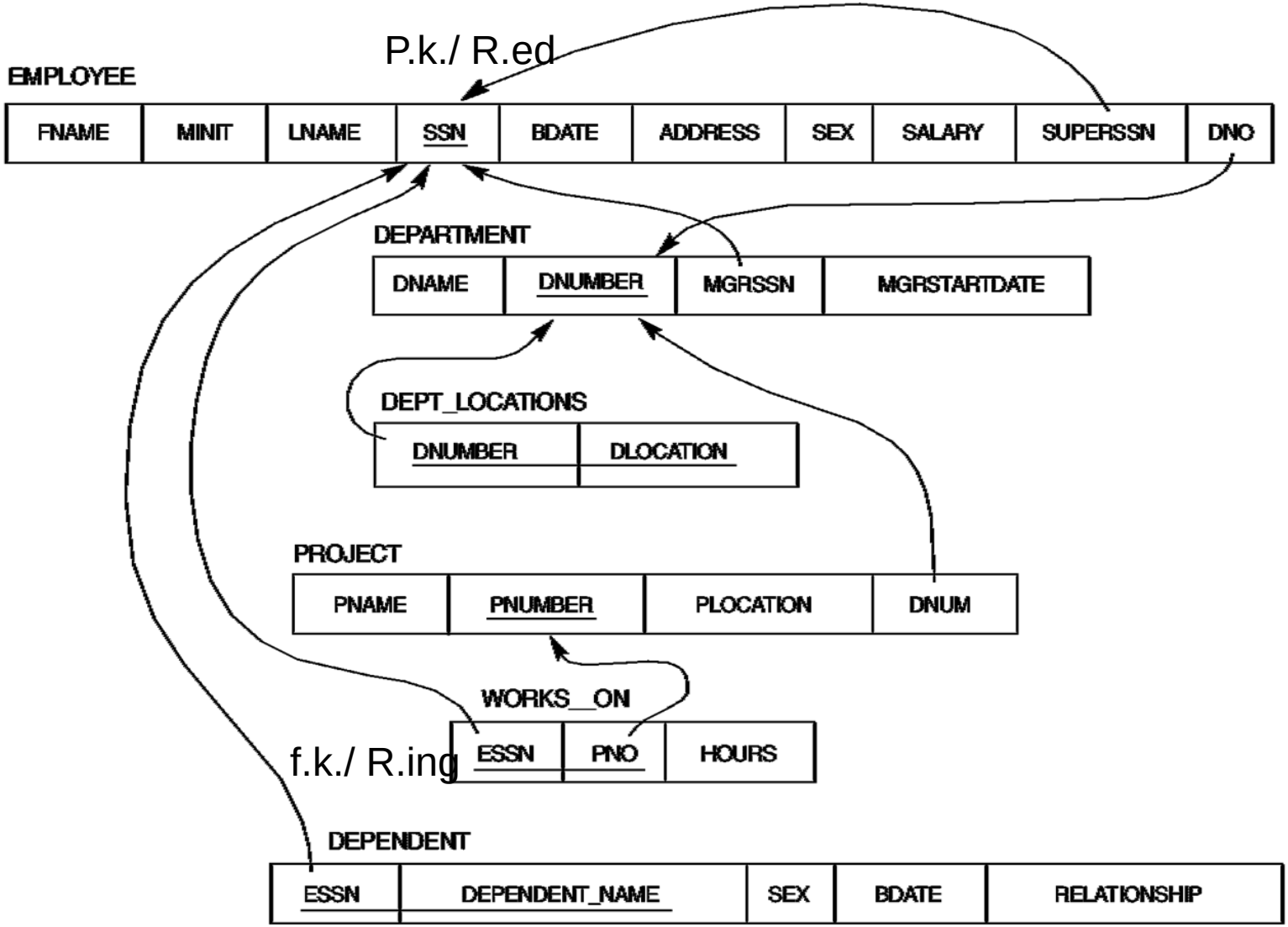
$t[PK] \neq null$ for any tuple t in r(R)

Referential Integrity

- A constraint involving *two* relations (the previous constraints involve a *single* relation).
- Used to specify a *relationship* among tuples in two relations: the **referencing relation** and the **referenced relation**.
- Tuples in the *referencing relation* R_1 have attributes FK (called **foreign key** attributes) that reference the primary key attributes PK of the *referenced relation* R_2. A tuple t_1 in R_1 is said to **reference** a tuple t_2 in R_2 if $t_1[FK] = t_2[PK]$.

  Note: the domain of FK has the same domain as the primary key attributes PK.
- A referential integrity constraint can be displayed in a relational database schema as a directed arc from R_1.FK to R_2.PK.

Referential Integrity Constraints displayed on the COMPANY relational schema diagram.



Referential Integrity Constraint

Statement of the constraint

The value in the foreign key column (or columns) FK of the the **referencing relation** R_1 can be either:

(1) a value of an existing primary key value of the corresponding primary key PK in the **referenced relation** R_2, or..

(2) a null.

In case (2), the FK in R_1 should not be a part of its own primary key.

Other Types of Constraints

Semantic Integrity Constraints:

- based on application semantics and cannot be expressed by the model.
- E.g., “the max. no. of hours per employee for all projects he or she works on is 56 hrs per week”
- A *constraint specification language* may have to be used to express these
- SQL-99 allows triggers and ASSERTIONS to allow for some of these

Figure: Schema diagram for the COMPANY relational database schema; the primary keys are underlined

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

DEPARTMENT

DNAME	DNUMBER	MGRSSN	MGRSTARTDATE

DEPT_LOCATIONS

DNUMBER	DLOCATION

PROJECT

PNAME	PNUMBER	PLOCATION	DNUM

WORKS_ON

ESSN	PNO	HOURS

DEPENDENT

ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP

Referential Integrity Constraints displayed on the COMPANY relational schema diagram.

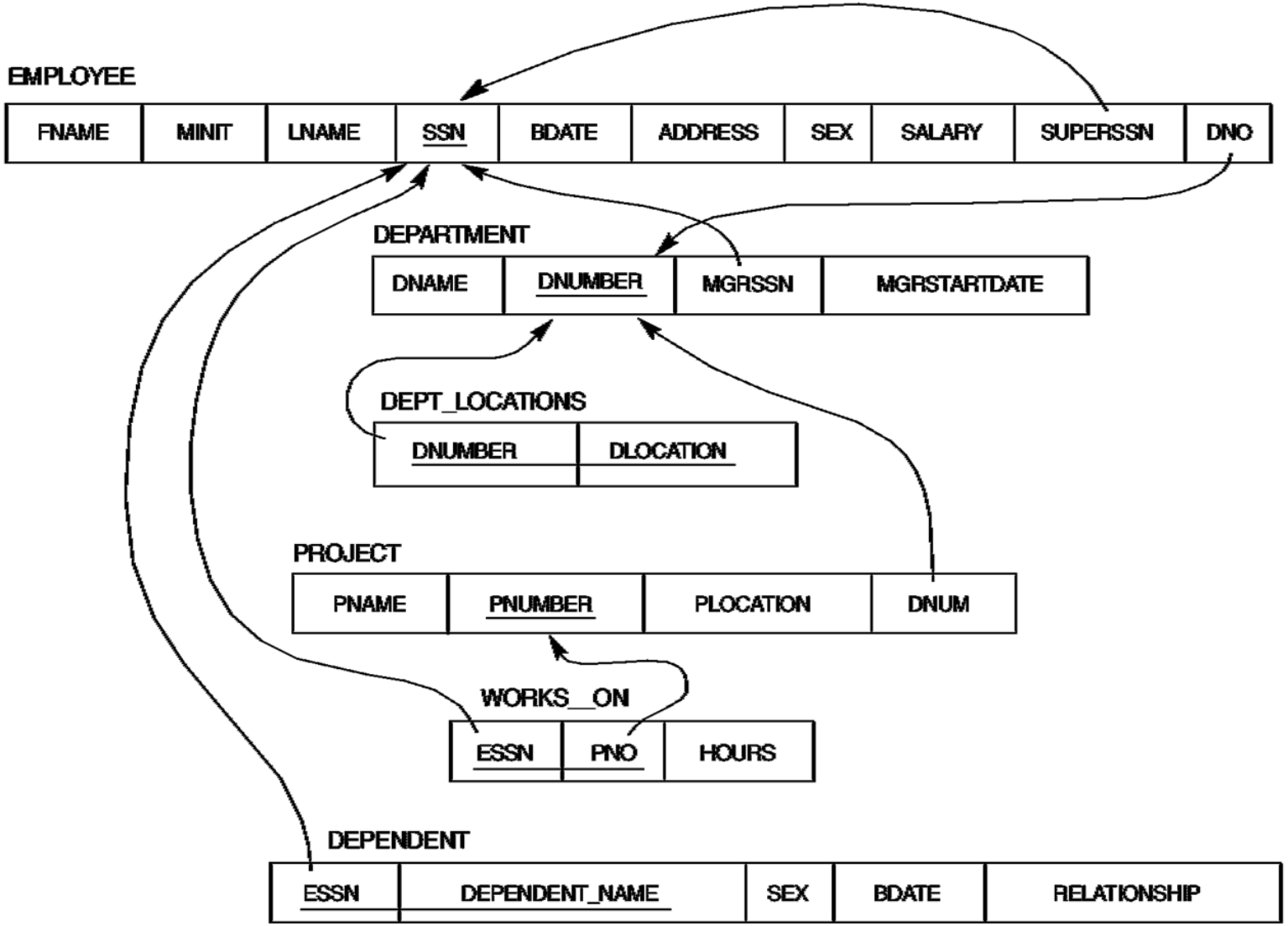


Figure: One possible relational database state corresponding to the COMPANY schema.

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John		Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin		Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia		Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer		Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh		Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce		English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad		Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James		Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
		Houston
		Stafford
		Bellaire
		Sugarland

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Update Operations on Relations

- INSERT a tuple.
- DELETE a tuple.
- MODIFY a tuple.

- Integrity constraints should not be violated by the update operations.
- Several update operations may have to be grouped together.
- Updates may *propagate* to cause other updates automatically. This may be necessary to maintain integrity constraints.

Update Operations on Relations

- In case of integrity violation, several actions can be taken:
  - Cancel the operation that causes the violation (REJECT option)
  - Perform the operation but inform the user of the violation
  - Trigger additional updates so the violation is corrected (CASCADE option, SET NULL option)
  - Execute a user-specified error-correction routine

Insert operation

discuss the following

- Insert <‘Cecilia’, ‘F’, ‘Kolonsky’, null, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, null, 4> into **`EMPLOYEE`**
- Insert <‘Alicia’, ‘J’, ‘Zelaya’,999887777, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, 987654321, 4 > into **`EMPLOYEE`**
- Insert <‘Cecilia’, ‘F’, ‘Kolonsky’, 677678989, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, null, 7> into **`EMPLOYEE`**

Delete operation
discuss the following

- Delete the **WORKS_ON** tuple with **ESSN** = ‘999887777’ and **PNO** = 10
- Delete the **EMPLOYEE** tuple with **SSN** = ‘999887777’
- Delete the **EMPLOYEE** tuple with **SSN** = ‘333445555’

Update operation

discuss the following

- Update the **SSLARY** of the **EMPLOYEE** tuple with **SSN** = ‘999887777’ to 28000
- Update the **DNO** of the **EMPLOYEE** tuple with **SSN** = ‘999887777’ to 7.
- Update the **SSN** of the **EMPLOYEE** tuple with **SSN** = ‘999887777’ to ‘987654321’.

In-Class Exercise

(Taken from Exercise 5.15)

Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:

STUDENT(<u>SSN</u>, Name, Major, Bdate)

COURSE(<u>Course#</u>, Cname, Dept)

ENROLL(<u>SSN</u>, <u>Course#</u>, <u>Quarter</u>, Grade)

BOOK_ADOPTION(<u>Course#</u>, <u>Quarter</u>, Book_ISBN)

TEXT(<u>Book_ISBN</u>, Book_Title, Publisher, Author)

Draw a relational schema diagram specifying the foreign keys for this schema.

Chapter 6

Functional Dependencies and Normalization for Relational Databases

Chapter Outline

1 Informal Design Guidelines for Relational Databases

- 1.1Semantics of the Relation Attributes
- 1.2 Redundant Information in Tuples and Update Anomalies
- 1.3 Null Values in Tuples
- 1.4 Spurious Tuples

2 Functional Dependencies (FDs)

- 2.1 Definition of FD
- 2.2 Inference Rules for FDs
- 2.3 Equivalence of Sets of FDs
- 2.4 Minimal Sets of FDs

Chapter Outline(contd.)

3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

4 General Normal Form Definitions (For Multiple Keys)

5 BCNF (Boyce-Codd Normal Form)

1 Informal Design Guidelines for Relational Databases (1)

- What is relational database design?

  The grouping of attributes to form "good" relation schemas
- Two levels of relation schemas
  - The logical "user view" level
  - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

Informal Design Guidelines for Relational Databases (2)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
  - 1NF (First Normal Form)
  - 2NF (Second Normal Form)
  - 3NF (Third Normal Form)
  - BCNF (Boyce-Codd Normal Form)